



Fish Quality Classification Project

Comprehensive Experiment Review

Sowmya Maddala • 22.10.2024

Project Overview

Objective:

Classify fish images into "Good" or "Bad" categories based on the quality

Species Focus:

Sardine

Motivation:

Improve model performance for app testing

Data Collection and Preparation

Data Sources

Daily data collected from S3 and manual data till Oct-8-2024, uploaded to Kaggle as [private dataset](#)

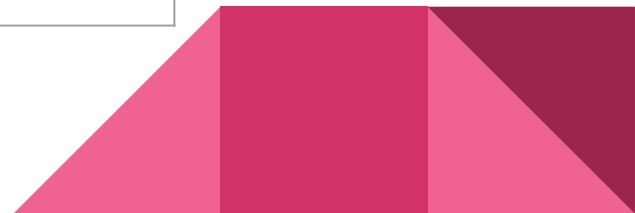
Dataset Segmentation

- Segmented and organized “single” fish image data into ‘Good’ and ‘Bad’ folders in [Kaggle](#).
- 500 images of the app testing data per class has been included in training and validation datasets as they are 3 fish “group” images.



Data Distribution

Dataset	Good	Bad
Training	5968	6010
Validation	1869	1873
Test	1712	1711
Unseen	50	51
App testing data	1423	670



Experiment 1: Initial Model Training

Model Architecture:

- Base Model: DenseNet121
- Layers: Global Average Pooling 2D, Dense (128 neurons), Dropout (20%), Dense (32 neurons), Sigmoid Output.

Framework and Configurations:

- TensorFlow Version: 2.15
- Training for 50 epochs, batch size of 32, input size 224x224 without data augmentation
- Callbacks: ReduceLROnPlateau, EarlyStopping.

Results:

- Training Accuracy: 77%
- Validation Accuracy: 74%
- Testing Accuracy: 73%
- Unseen Data Accuracy: 76%

Insights: Initial model showed good promise but indicated overfitting.



Experiment 2: Adding Regularization and More Layers

Updated Model Architecture:

- Base Model: DenseNet121
- Additional Layers: Global Average Pooling 2D, Dense (512 neurons, L2 regularization 0.0001), Dense (128 neurons), Dropout (20%), Dense (32 neurons), Sigmoid Output.

Framework and Configurations:

- TensorFlow Version: 2.10
- Training Configuration: Same as before, but training stopped early at the 18th epoch due to EarlyStopping.

Results:

- Training Accuracy: 82%
- Validation Accuracy: 76%
- Test Accuracy: 76%
- Unseen Data Accuracy: 80%

Challenges Faced:

- App testing data predictions were biased towards the "Good" class.
- Hypothesized that the lack of segmented group fish images in training data contributed to bias.

Experiment 3: Increased neurons

Updated Model Architecture:

- Base Model: DenseNet121
- Layers: Global Average Pooling 2D, Dense (512 neurons, L2 regularization), Dropout (20%), Dense (**256 neurons**), Dense (32 neurons), Sigmoid Output.

Framework and Configurations:

- TensorFlow Version: 2.10
- Training Configuration: Same as before

Results:

Similar performance; training and validation metrics did not improve significantly.



Experiment 4: Batch Normalization and additional fine-tuning

Modifications

- New Dataset Version: Included segmented images of group fish (500 images per class for training and validation) from app testing data in training and validation data.
- Incorporated Batch Normalization:
 - a. Significance: Normalizes outputs for faster training and better performance.
 - b. Enhanced model stability and convergence.
- Architecture:
 - a. Included Batch Normalization layers after dense layers.
 - b. Applied **L2 Regularization** in the **output layer** to prevent overfitting.
- Further Model Changes: Enabled all layers of the base model for training (fine-tuning).

Experiment 4: Batch Normalization and additional fine-tuning

Framework:

- TensorFlow Version: 2.16 (model loading issue fixed).

Training Configuration:

- Continued training from saved weights (previous 50-epoch model).
- Learning Rate: Lowered to $1e-5$.
- Ran for 20 epochs with data augmentation.

Performance Results:

- Training Accuracy: 88%
- Validation Accuracy: 81%
- Test Accuracy: 83%
- Unseen Data Accuracy: 86%
- App Testing Data Accuracy: 81%

Insights: The introduction of batch normalization significantly boosted performance metrics.



Experiment 5: Exploring Other Base Models

1 Pre-trained models tested	2 Learning Rate Schedulers	3 Model Weights Used
• MobileNet V2 • DenseNet 121 • DenseNet 169 • DenseNet 201 • VGG 16 • VGG 19 • Resnet	Compared Exponential Decay vs. ReduceLROnPlateau, with ReduceLROnPlateau being most effective.	Both checkpoint and final weights are viable options for fine-tuning, but checkpoint weights can help resume training from an earlier point in case of interruptions.

Results Summary

Pre-trained Model	Validation Accuracy	Test Accuracy	App testing accuracy
MobileNet V2	83	80	79
DenseNet 121	81	83	81
DenseNet 169	79	81	79
DenseNet 201	80	82	80
VGG 16	81	77	76
VGG 19	81	79	80
Resnet	79	75	77

Current Best Performing Model Architecture

Layer (type)	Output Shape	Param #
densenet121 (Functional)	(None, 7, 7, 1024)	7,037,504
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1024)	0
dense (Dense)	(None, 512)	524,800
batch_normalization (BatchNormalization)	(None, 512)	2,048
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
batch_normalization_1 (BatchNormalization)	(None, 256)	1,024
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 1)	257

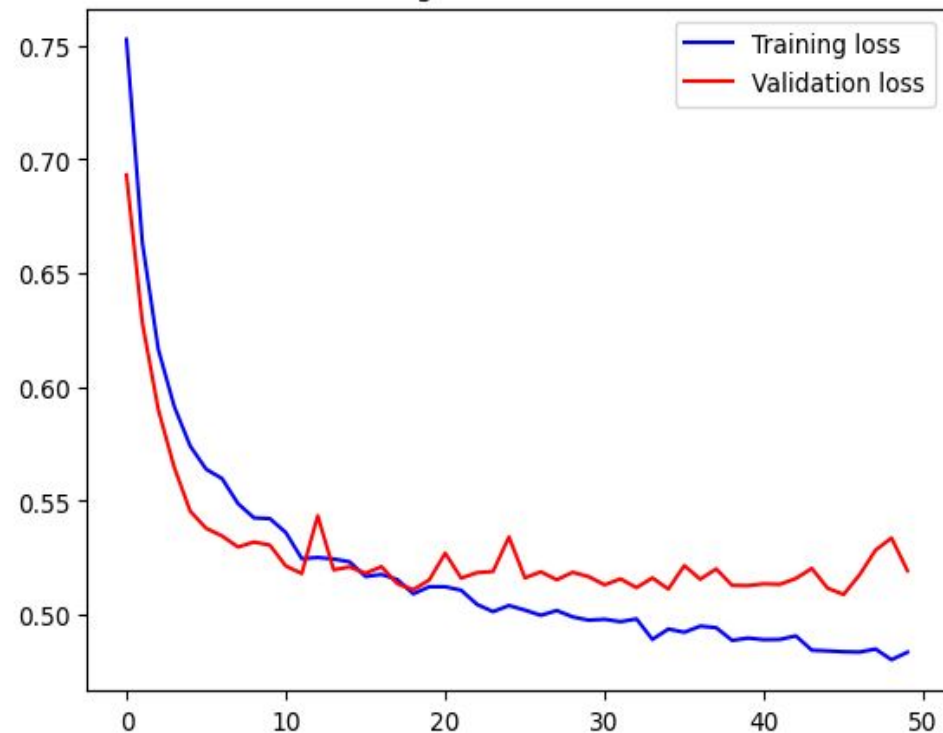
Confusion Matrices Analysis

- Achieved **86%** accuracy on October 8-20 data collected.
- Link to Confusion Matrices Presentation: [[Link](#)]

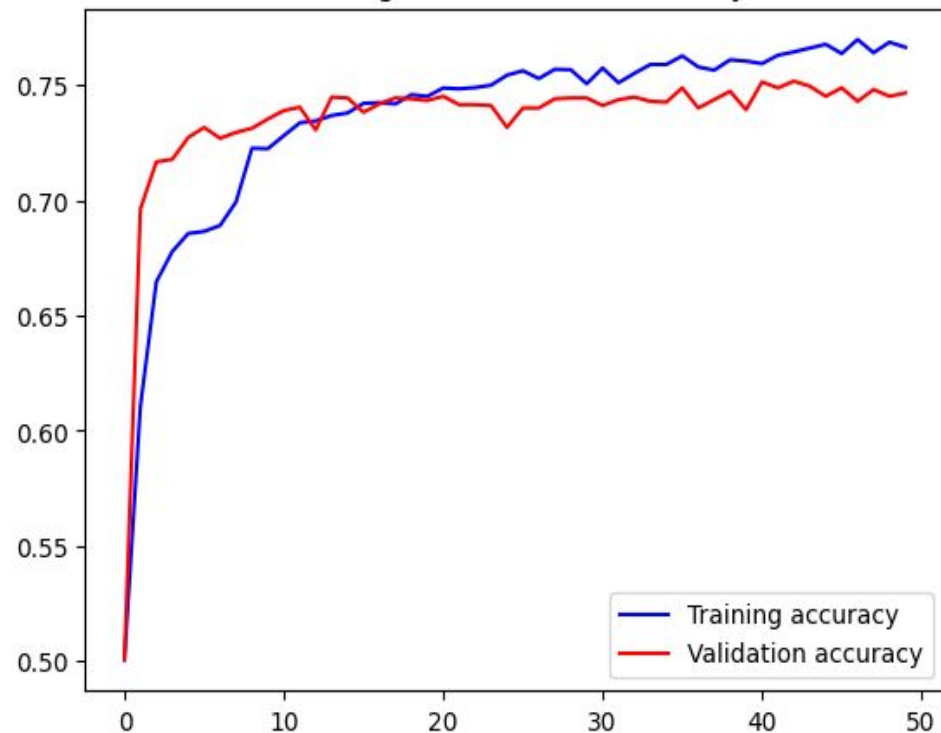


Training Accuracy Loss Plots

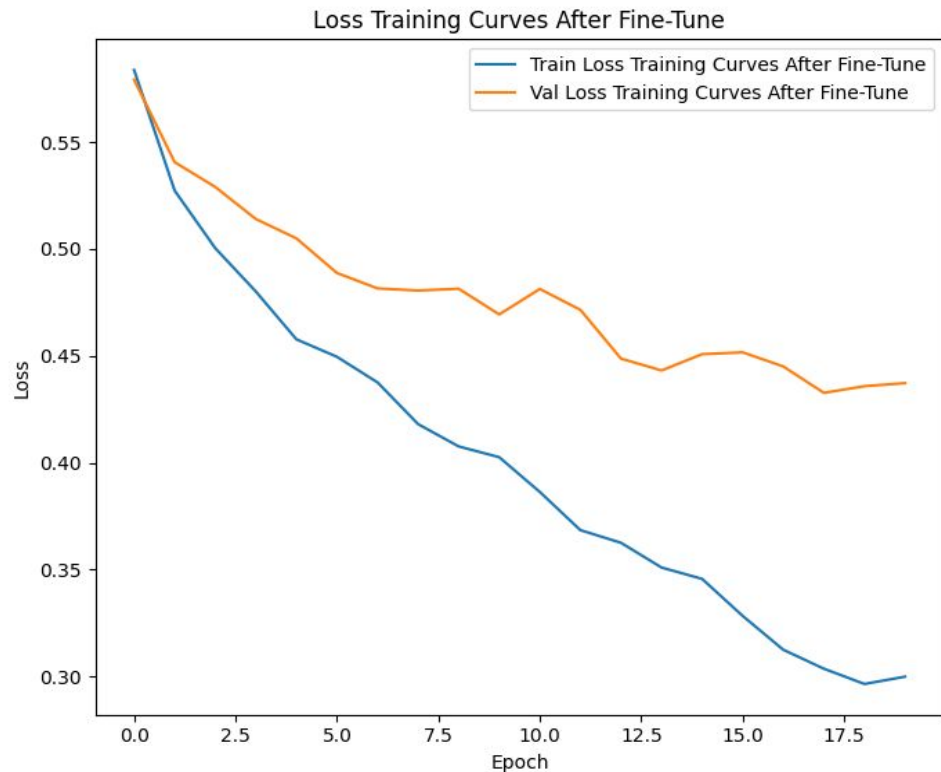
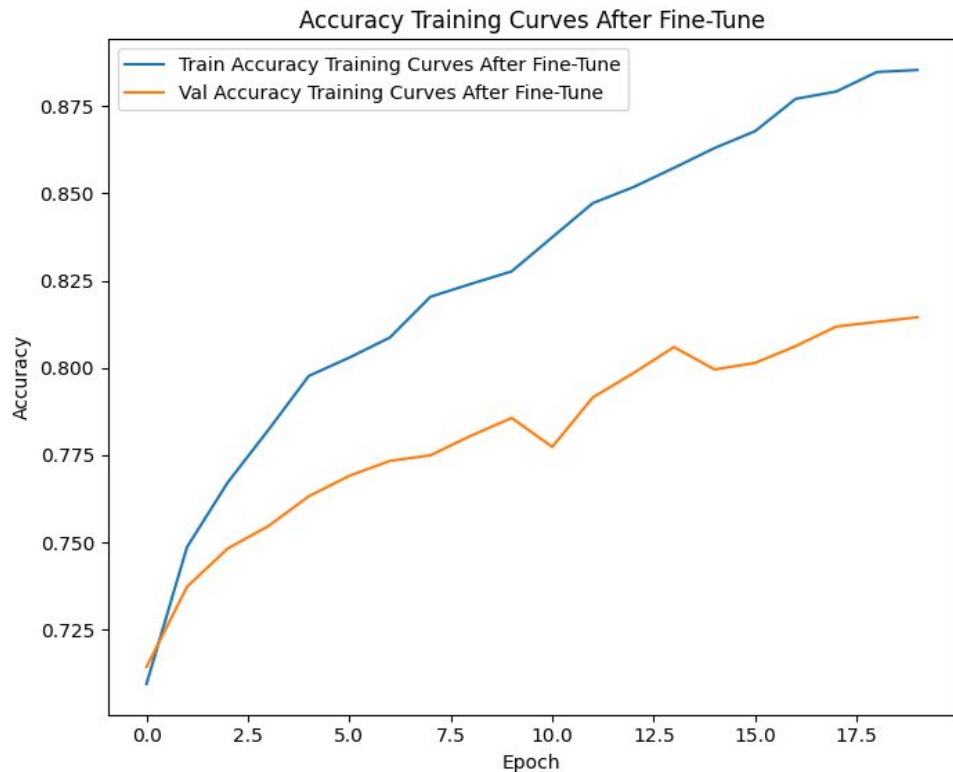
Training and validation loss



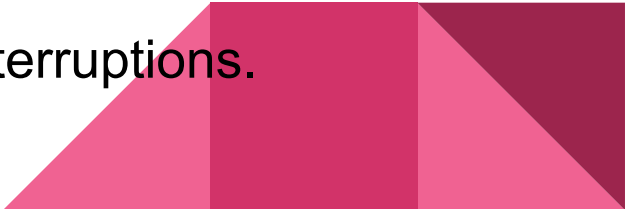
Training and validation accuracy



Training Accuracy Loss Plots of fine-tuning



Challenges Faced

- **Initial Issues:**
 - Older TensorFlow versions gave errors; switched to 2.16.
 - Saving the models in tensorflow SavedModel format.
 - Managing storage for segmented datasets.
 - **Biased Results:**
 - Challenges with app testing data (biased towards "Good").
 - Resolution: Included segmented group fish images for better generalization.
 - Internet connectivity issues causing training interruptions.
- 

Conclusion

Model Performance

- DenseNet121 outperformed others, achieving **83% testing accuracy** and **82% app testing accuracy**.
- **Fine-tuning** this model significantly enhanced its performance for our specific dataset.

Batch Normalization

This technique accelerated training and enhanced accuracy, proving essential in our deep learning approach.

Regularization Impact

The inclusion of L2 regularization, especially at output layer, and dropout layers significantly improved performance by reducing overfitting.

Future Work

1. Explore various machine learning models to further improve accuracy.
 2. Use the best-performing model as a feature extractor.
 3. Implement PCA and LDA for dimensionality reduction on a larger dataset.
 4. Aim to enhance model robustness through iterative testing and adjustments.
-



Thank You!